

that 49 is too high and gives you another chance to guess. You could guess 48, then 47, and so on until you get the right answer. But that could take a long time—if the correct number is 1, it will take you 50 guesses to get it, which seems like too many guesses considering there were only 100 total possibilities to begin with.

A better approach is to take larger jumps after you find out whether your guess is too high or too low. If 50 is too high, consider what we could learn from guessing 40 next instead of 49. If 40 is too low, we have eliminated 39 possibilities (1–39) and we’ll definitely be able to guess in at most 9 more guesses (41–49). If 40 is too high, we’ve at least eliminated 9 possibilities (41–49) and we’ll definitely be able to guess in at most 39 more guesses (1–39). So in the worst case, guessing 40 narrows down the possibilities from 49 (1–49) to 39 (1–39). By contrast, guessing 49 narrows down the possibilities from 49 (1–49) to 48 (1–48) in the worst case. Clearly, guessing 40 is a better searching strategy than guessing 49.

It turns out that the best searching strategy is to guess exactly the midpoint of the remaining possibilities. If you do that and then check whether your guess was too high or too low, you can always eliminate half of the remaining possibilities. If you eliminate half of the possibilities in each round of guessing, you can actually find the right value quite quickly ($O(\log(n))$ for those keeping score at home). For example, a list with 1,000 items will require only 10 guesses to find any element with a binary search strategy. If we’re allowed to have only 20 guesses, we can correctly find the position of an element in a list with more than a million items. Incidentally, this is why we can write guessing-game apps that can correctly “read your mind” by asking only about 20 questions.

To implement this in Python, we will start by defining upper and lower bounds for what location a file can occupy in a filing cabinet. The lower bound will be 0, and the upper bound will be the length of the cabinet:

```
sorted_cabinet = [1,2,3,4,5]
upperbound = len(sorted_cabinet)
lowerbound = 0
```

To start, we will guess that the file is in the middle of the cabinet. We’ll import Python’s *math* library to use the `floor()` function, which can convert